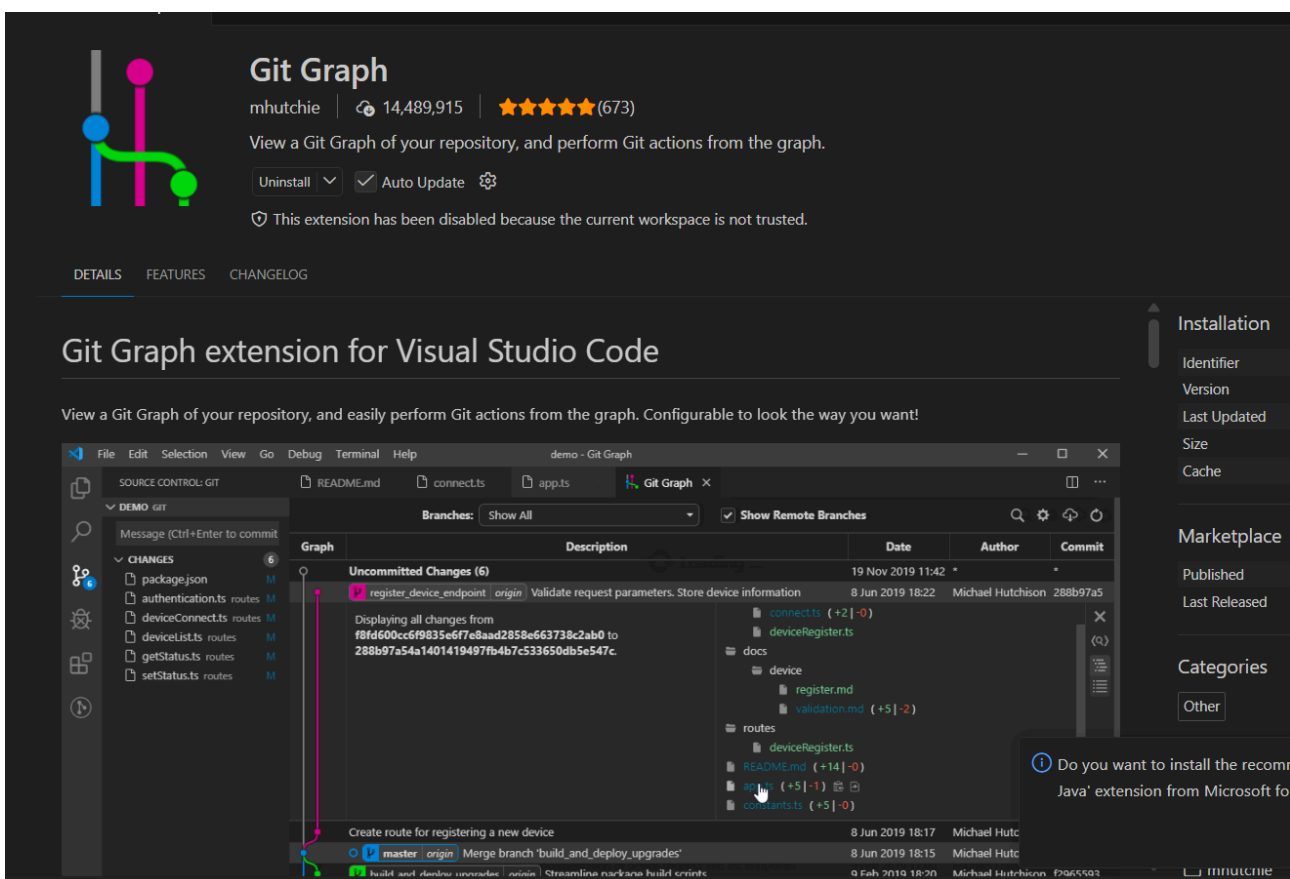
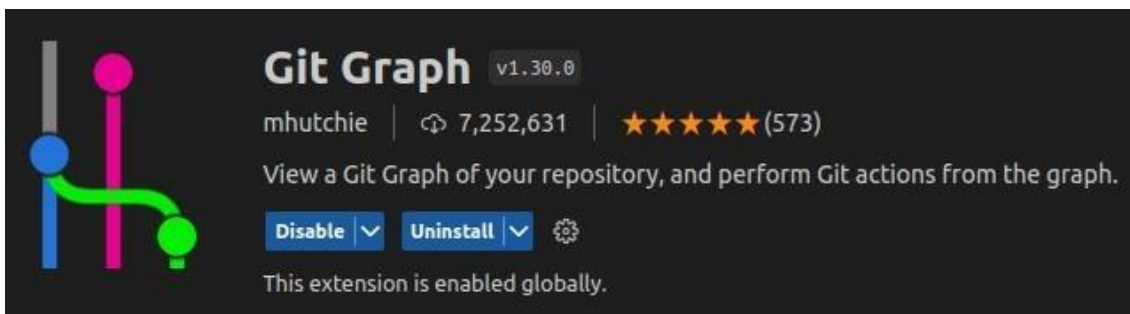


Práctica Guiada: Empezando a trabajar con repositorios remotos en Github

En esta práctica vamos a empezar a trabajar con repositorios remotos, concretamente con repositorios remotos alojados en GitHub.

Antes de empezar con la práctica:

A la hora de trabajar con ramas de VS Code es muy útil el plugin “Git Graph”:



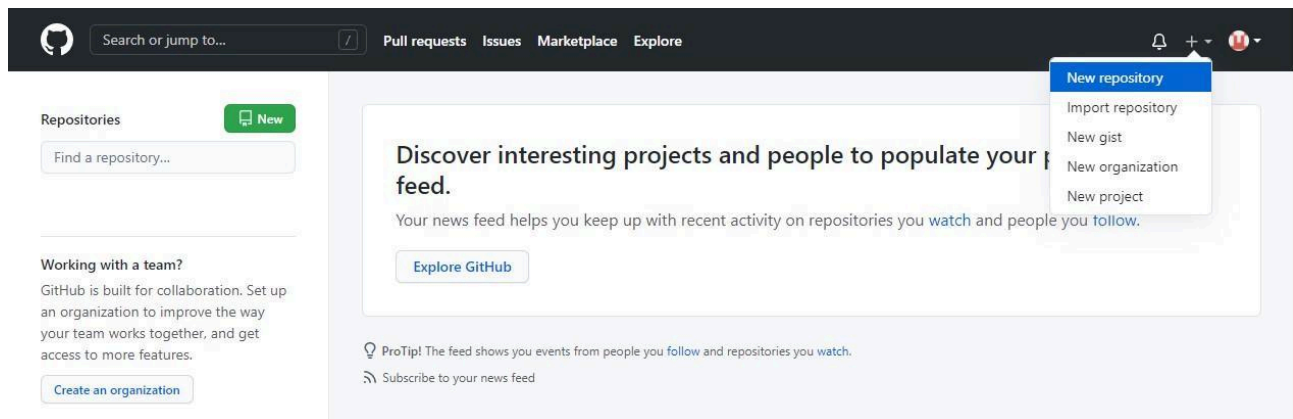
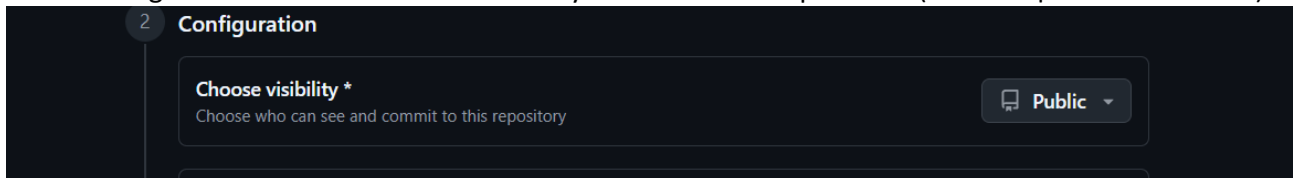
Este plugin te representa las distintas ramas de tu control de versiones con los commits que has hecho de forma gráfica. También te permite comparar contenido de distintas versiones con ayuda visual.

1. Cuenta en Github y creando nuestro repositorio

Nota: Se recomienda hacer la práctica con Visual Studio code y su terminal ya que facilita el trabajar con GitHub, especialmente la parte de las autenticaciones cuando trabajemos para actualizar rama remotas.

Lo primero es ir a la pagina oficial de [GitHub](https://github.com) y crear una cuenta si no la tienes ya.

Una vez tengas la cuenta creada entra en ella y crea un nuevo repositorio (crea el repositorio PUBLICO)



- 1.1. Después de crear el repositorio indica aquí la URL de tu repositorio para poder visitarla: URL Repo GitHub: XXXXXX

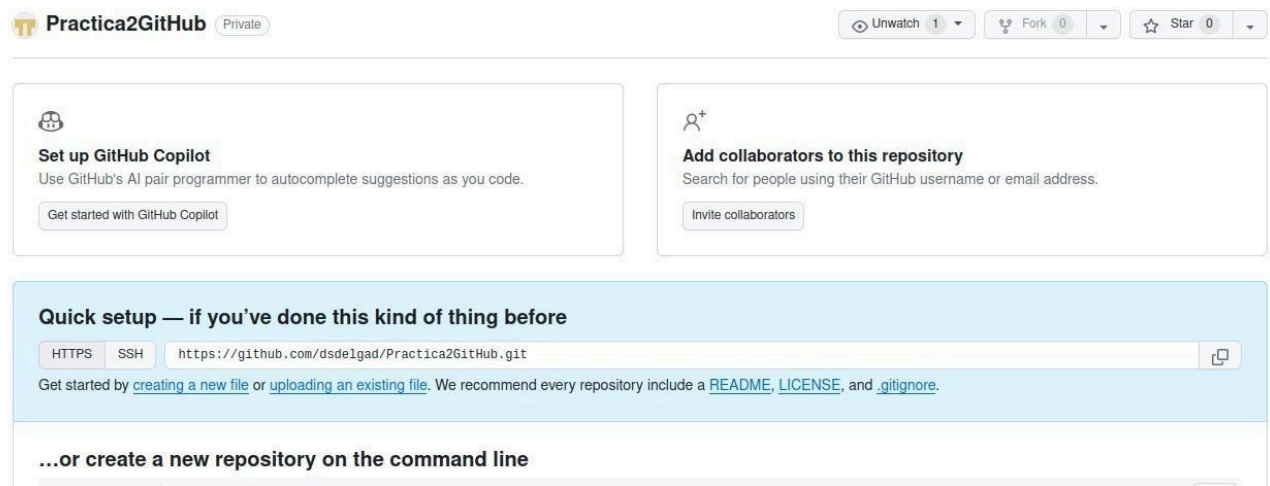
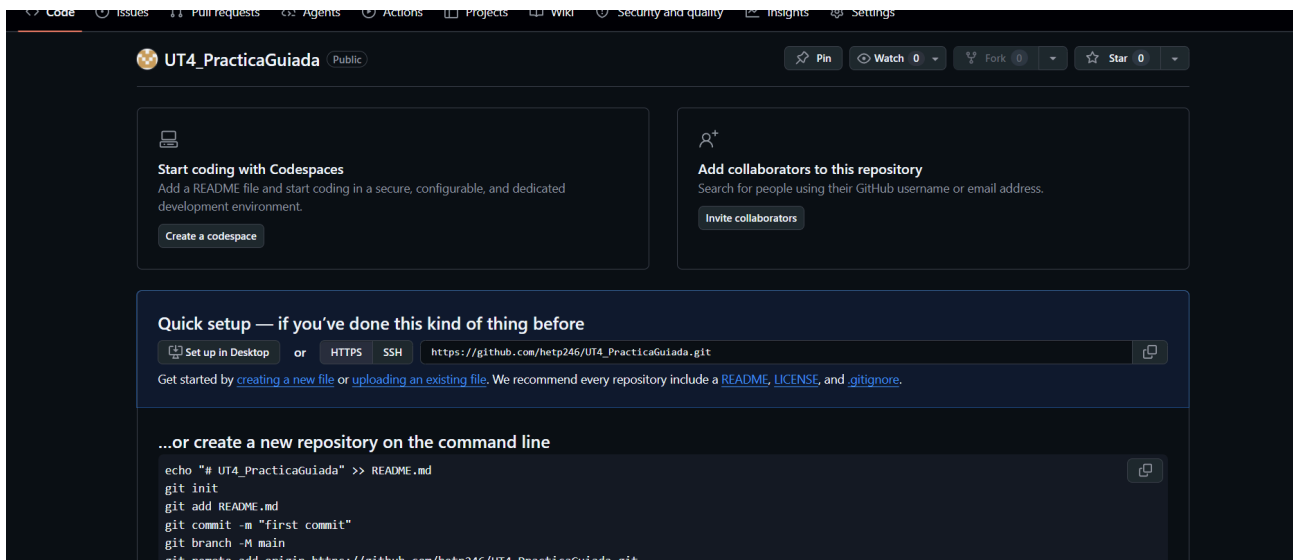
GitHub: https://github.com/hetp246/UT4_PracticaGuiada.git

2. Vinculando repositorios remotos y empezando a trabajar.

Importante: Para esta parte de la práctica vamos a partir de un repositorio con control de versiones que ya tengamos en Local. Te recomiendo que utilices el **mismo** repositorio que creaste para la **práctica anterior**. (Si no quieres hacer esto, puedes crear un repo local como vimos en la práctica anterior y hacer algunos commits para registrar en control de versiones).

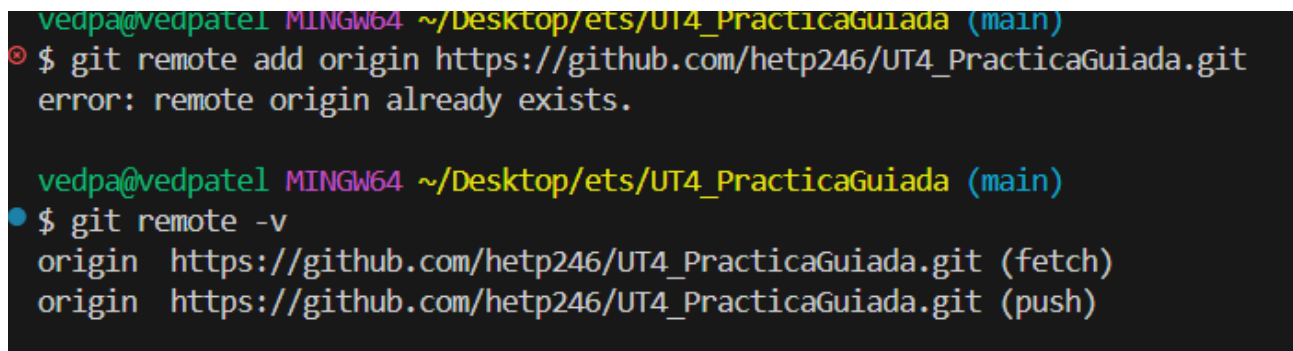
Si queremos trabajar y contribuir a un repositorio remoto, **lo primero que tenemos hacer es vincularlo** nuestro entorno **local**.

Para ello, una vez creado el repo, en GitHub vamos a la sección “code” y copiamos la url nuestro repo.



Y teniendo como directorio de trabajo actual, la carpeta donde está nuestro repo local, ejecutamos algo como esto:

```
git remote add origin <url_de_tu_repo>
```



Nota: En lugar de origin le puedes poner el nombre que quieras, se suele poner este por defecto. Pero si necesita vincular mas de un repositorio remoto tendrás que ponerles nombres distintos a cada vinculación para que Git sepa con qué repo trabajar.

Puedes comprobar los repos que tienes vinculados si ejecutas un: `git remote -v`

- **2.1. Ejecuta los comandos mencionados para vincular y comprobar que has vinculado correctamente tu repo.**

Nota: Si ejecutamos `git remote` veremos los repositorios remotos que tenemos vinculados ahora mismo. Y si quisiéramos dejar de vincular alguno, simplemente ejecutaríamos:

`git remote rm <nombre>`

A continuación vamos a **subir** los ficheros de nuestro repo local al repositorio remoto que hemos creado. Para ello ejecuta un: `git push` y observa el error que devuelve, a continuación lee el error y ejecuta la sugerencia de git

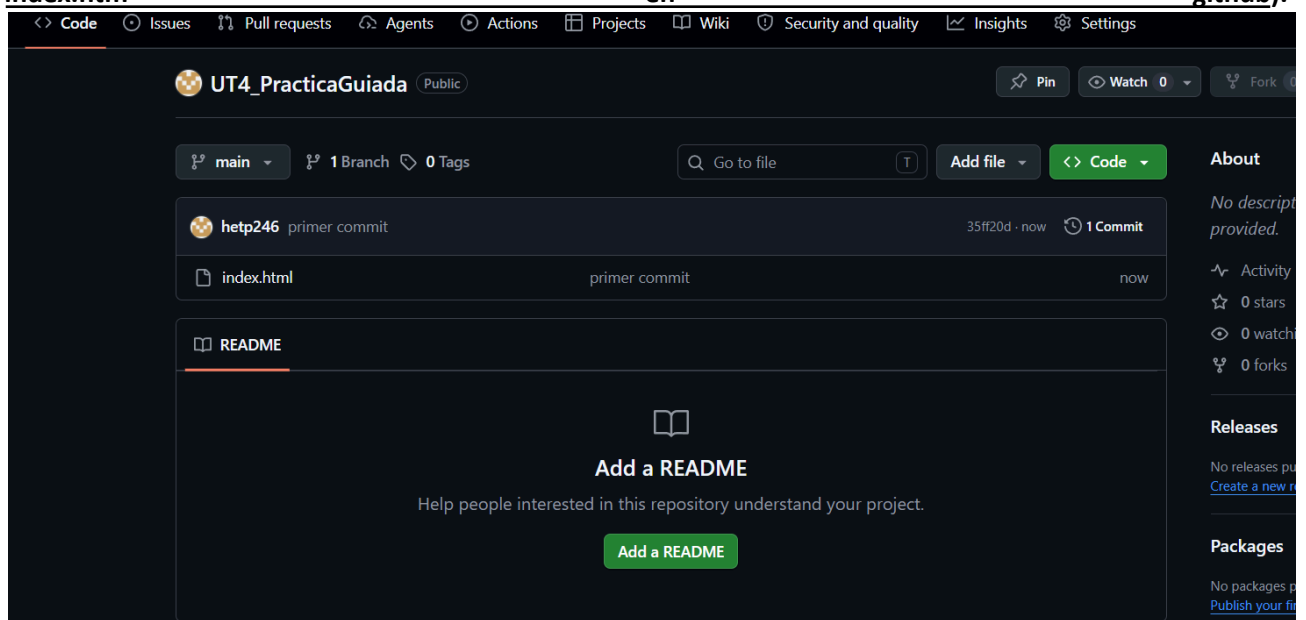
```
davidd@El-Equipo:~/Documentos/PythonProyectos/Docencia/ETS_1_DAW/PracticasGIT$ git push
fatal: La rama actual master no tiene una rama upstream.
Para realizar un push de la rama actual y configurar el remoto como upstream, usa

git push --set-upstream origin master
```

Al ser la primera vez que hacemos un push a este repositorio remoto nos dice que tenemos que especificar la branch a la que hacer los cambios.

- **2.2. Ejecuta el comando y adjunta captura de lo que devuelve, luego ve a github y observa como el repositorio ya no está vacío (Adjunta captura de esto es decir el comando y la vista del repo de GitHub).**

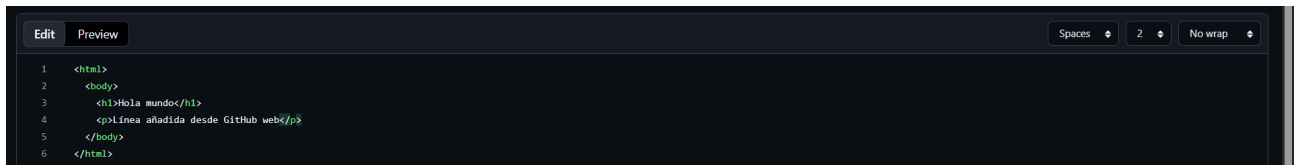
- **2.3. Haz una modificación al fichero index.html (la que quieras), registra el cambio en tu repositorio local y ejecuta: `git push`. Observa como ahora no te da error. Adjunta captura donde se vean los comandos que ejecutaste para registrar el cambio en el fichero local y el push; y comprueba que el cambio está en el repositorio remoto (resalta la línea nueva que introdujiste en la captura de pantalla del index.htm en github).**



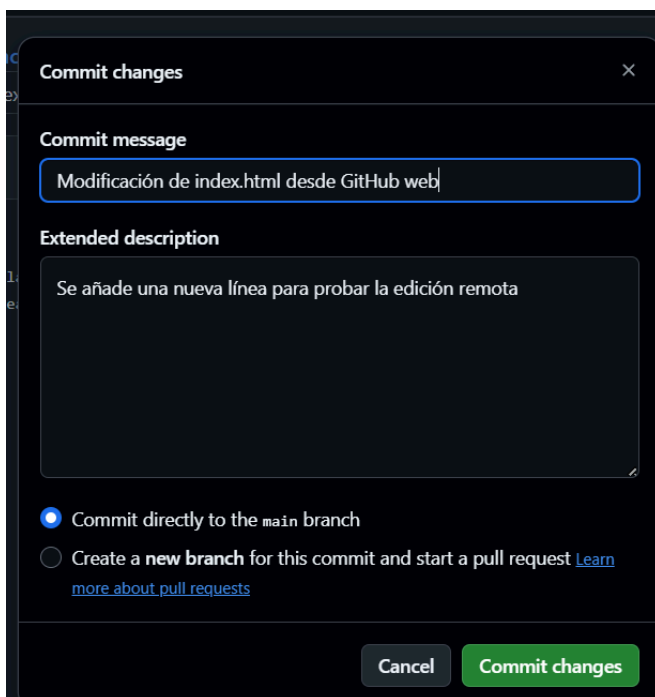
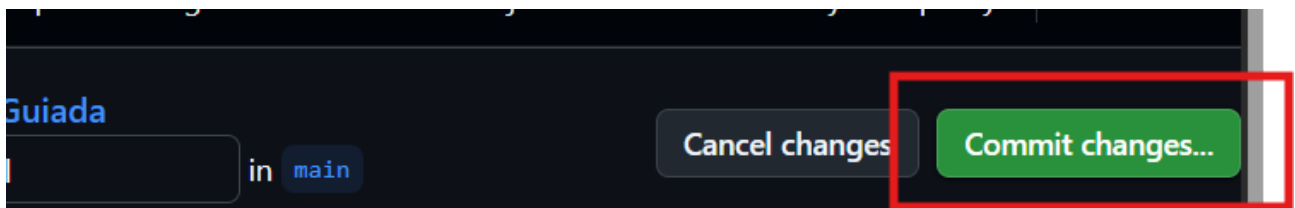
3. Editando código desde GitHub y Pull

Vamos a ver como se puede editar nuestro código desde el propio servicio web de GitHub y como funciona un **pull**.

Desde Github en el navegador web, accede al fichero index.html, modifícalo **añadiendo una nueva línea** (la que quieras, pero un contenido diferenciador con el resto del fichero) y haz el **commit**.



```
1 <html>
2 <body>
3   <h1>Hola mundo</h1>
4   <p>Línea añadida desde GitHub web</p>
5 </body>
6 </html>
```



Commit changes

Commit message

Modificación de index.html desde GitHub web

Extended description

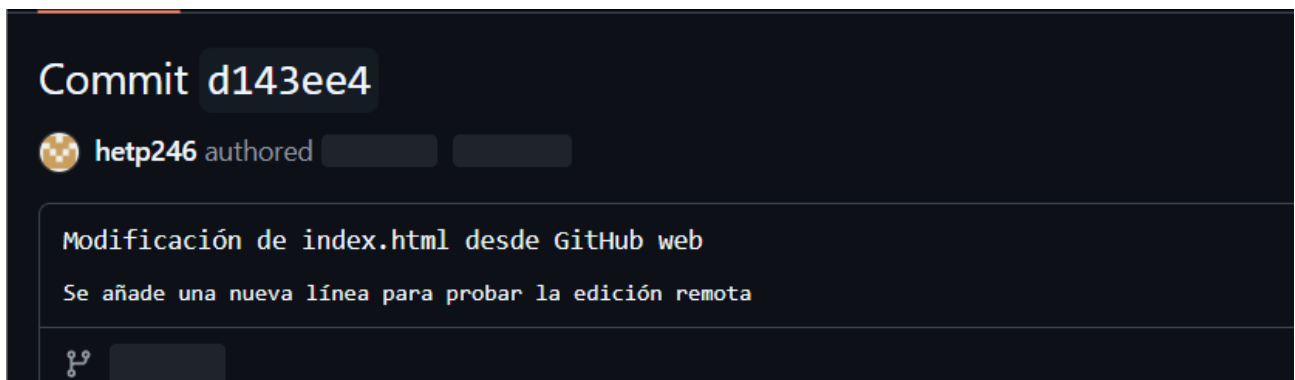
Se añade una nueva línea para probar la edición remota

☒ Commit directly to the main branch

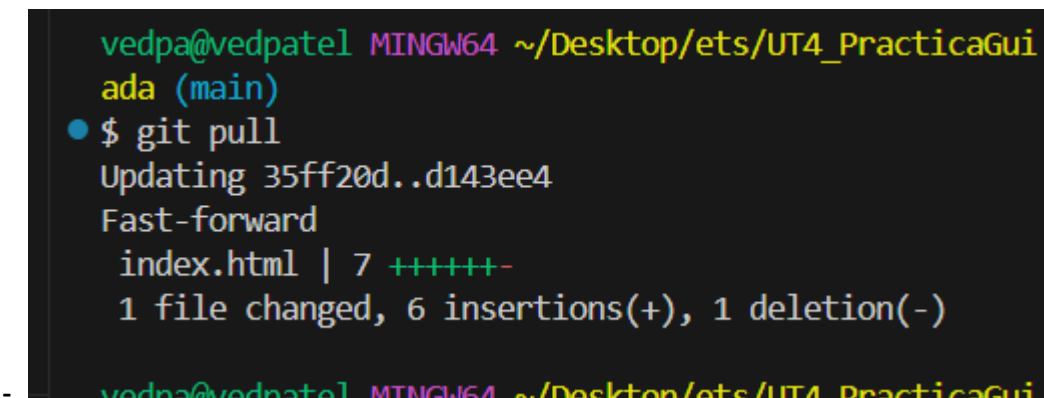
☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

Cancel Commit changes

- 3.1. Adjunta captura de la modificación desde la interfaz web de Github(Ponle un buen comentario al commit que describa lo que estás haciendo).

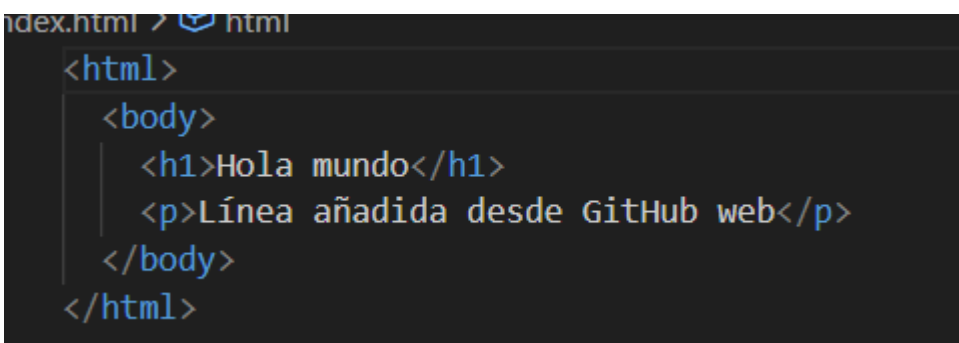


- 3.2. Ejecuta desde la terminal el comando: git pull



Adjunta capturas y comenta lo que te devuelve. Observa el fichero index.html en tu repositorio local y confirma que se ha actualizado tu repo con el último cambio.

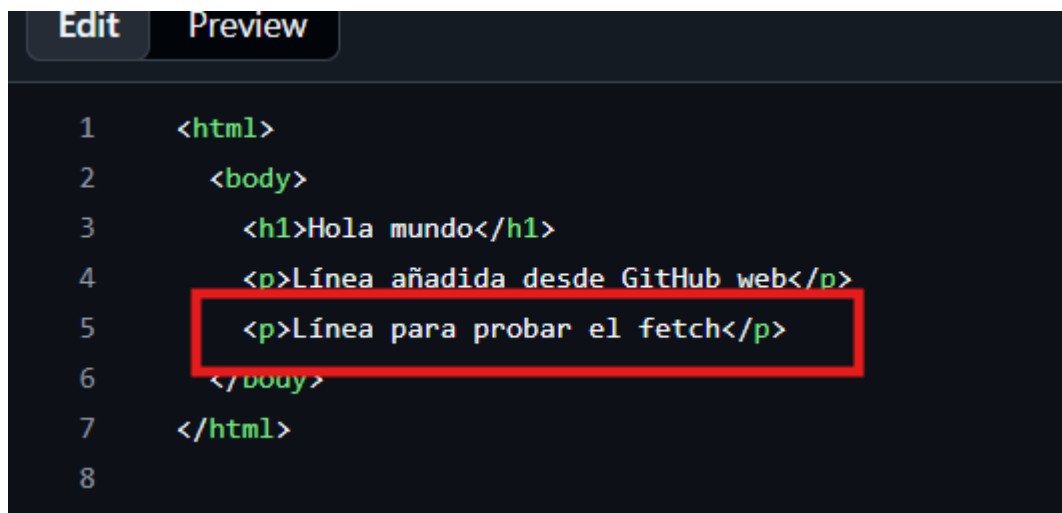
Si me a añadido la parte de <p>



4. Entendiendo el fetch y el pull

Cada vez que comiences a trabajar en un proyecto, debes obtener los cambios más recientes en tu copia local para asegurarte de estás trabajando en la última versión, con Git puedes hacer eso con el comando **pull** como hemos comentado en clase.

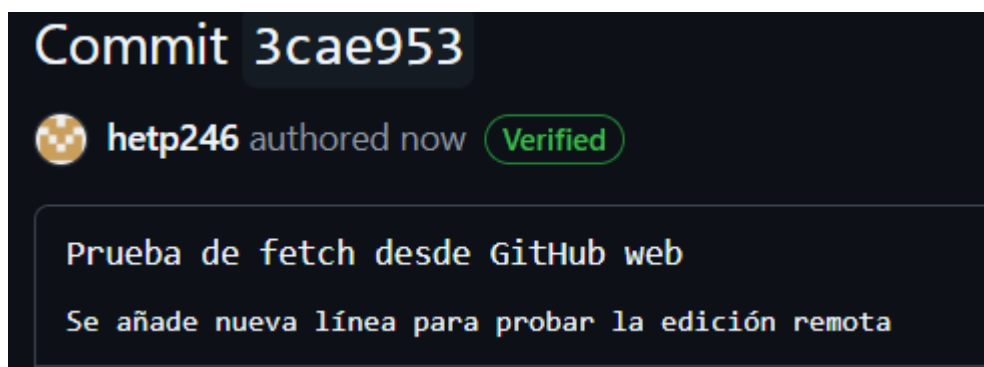
Un **fetch** no te modifica nada de tu repo local, solamente **obtiene** todo el historial de cambios de una rama/repositorio. Esto puede ser útil para si quieres solamente **comprobar** si hay cambios en el repo remoto, pero no tienes claro hacer el pull todavía (harías un fetch seguido de un git status).



The screenshot shows a code editor with two tabs: 'Edit' and 'Preview'. The 'Edit' tab is active, displaying an HTML file. The code is as follows:

```
1 <html>
2   <body>
3     <h1>Hola mundo</h1>
4     <p>Línea añadida desde GitHub web</p>
5     <p>Línea para probar el fetch</p>
6   </body>
7 </html>
8
```

The line `<p>Línea para probar el fetch</p>` on line 5 is highlighted with a red rectangular box.



The screenshot shows a GitHub commit interface. At the top, it says 'Commit 3cae953'. Below this, there is a profile picture of a person with the username 'hetp246' and the text 'authored now' next to a green 'Verified' badge. The commit message is 'Prueba de fetch desde GitHub web'. Below the message, there is a description of the changes: 'Se añade nueva línea para probar la edición remota'.

Un **pull** es una combinación de 2 comandos diferentes:

- Fetch

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
• $ git fetch

vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
• $ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)
```

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
• $ cat index.html
<html>
  <body>
    <h1>Hola mundo</h1>
    <p>Línea añadida desde GitHub web</p>
  </body>
</html>
```

- Merge

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
• $ git merge origin/main
Updating d143ee4..3cae953
Fast-forward
 index.html | 1 +
 1 file changed, 1 insertion(+)
```

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
• $ cat index.html
<html>
  <body>
    <h1>Hola mundo</h1>
    <p>Línea añadida desde GitHub web</p>
    <p>Línea para probar el fetch</p>
  </body>
</html>
```


Un **merge** es una **fusión** de dos ramas y ya hemos visto como funciona.

Vamos a verlo mejor con un ejemplo.

- **4.1. Vuelve a modificar el fichero index.html desde github y en lugar de hacer un pull, ejecuta un: git fetch seguido de un git status. Adjunta captura y comenta lo que pasa. ¿Ha cambiado tu index.html local?**

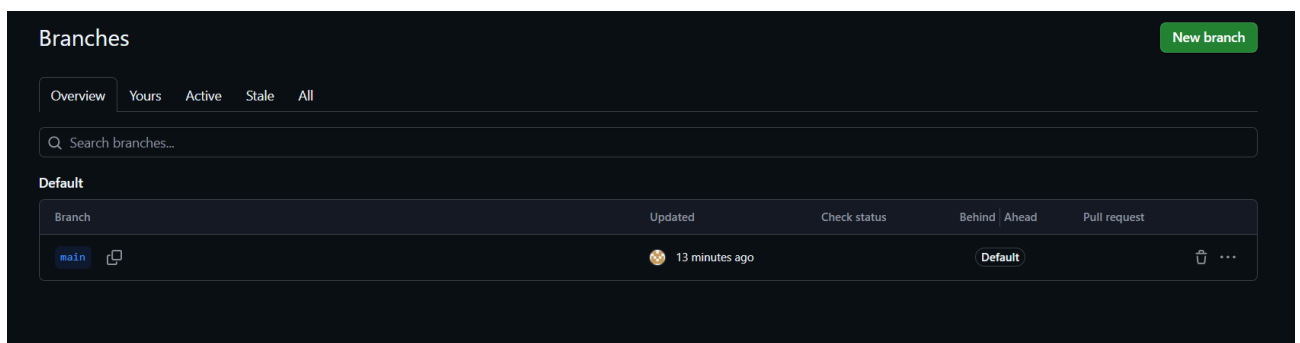
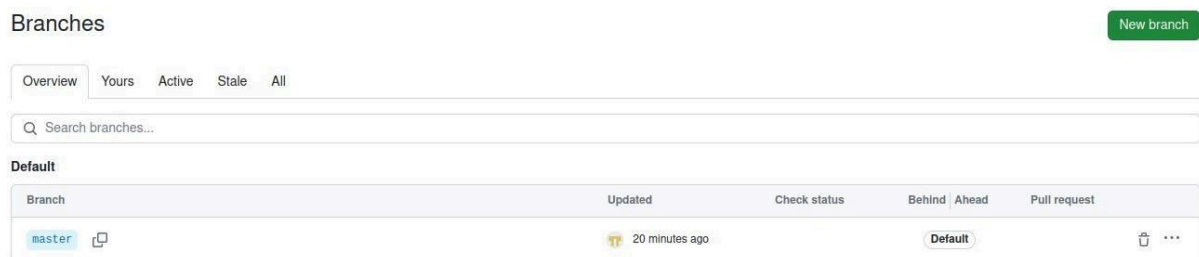
Es exactamente lo mismo que ya hice antes

- **4.2. Ejecuta un: git merge origin/master Y comenta lo que ha ocurrido.**

Es exactamente lo mismo que ya hice antes

5. Creando ramas en GitHub:

En **GitHub**, accede a tu repositorio y haz clic en el botón "master" para acceder a la rama y haz click en la opción de **“View all branches”**. Llegarás a un pantalla como esta:

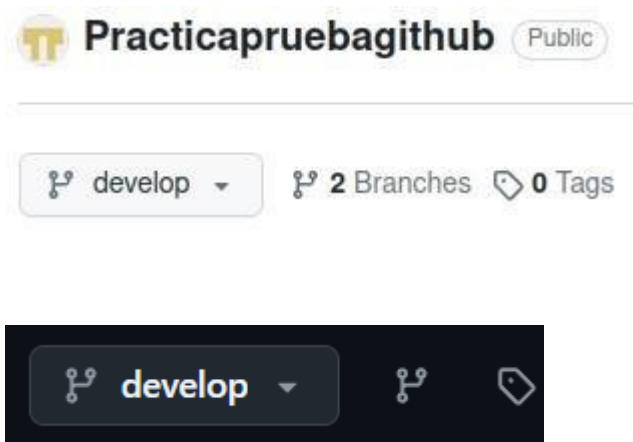


Ahí puedes crear una nueva rama.

- **5.1. Crea una nueva rama y adjunta captura de la rama creada en github.**

Default					
Branch	Updated	Check status	Behind	Ahead	Pull request
main	14 minutes ago			Default	
Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
develop	now		0	0	
Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
develop	now		0	0	

- 5.2. Muévete a la rama que acabas de crear (es decir selecciona esa nuevarama en la interfaz web):





Después modifica el fichero index.htm desde el editor web de github y haz un nuevo commit, pero esta vez en la rama que acabas de crear.

- 5.3. Ahora desde la terminal de tu equipo local (en VS Code o Git Bash) vuelve ejecutar un git pull. Adjunta captura de lo que devuelve y comenta lo que está pasando.

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
• $ git pull
Already up to date.
```

Puedes ver todas las ramas que tienes vinculadas si ejecutas un: **git branch -a**

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
• $ git branch -a
* main
remotes/origin/HEAD -> origin/main
remotes/origin/develop
remotes/origin/main
```

- 5.4. Ejecuta el comando anterior e identifica las dos ramas remotas que tienes.

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
● $ git branch -a
  * main
    remotes/origin/HEAD -> origin/main
    remotes/origin/develop
    remotes/origin/main
```

Nota: Si solo quieres ver las ramas remotas: **git remote-r**

- 5.5. Ejecuta el comando:

git checkout <nombre_nueva_rama_remota>

Adjunta captura y explica lo que te ha devuelto la terminal tras ejecutar el comando.

IMPORTANTE ANTE DE EJECUTAR COMANDO: <nombre_nueva_rama_remota> no

le pongas "remotes/origin/" delante ya que lo que vas a hacer es crear una ram nueva en local con el contenido de la remota.

Switched to a new branch 'develop'

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (develop)
● $ git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.
```

- 5.6. Ejecuta un: **git branch**. ¿En que rama estás ahora? Adjunta captura.

¡Enhorabuena! Has hecho una copia de la rama remota en local. Ahora si editas el fichero desde local y haces un **commit** y luego un **git push**, deberías actualizar la rama remota que has creado, no la master. Haz la prueba. Es decir:

solo las palabras **git branch** y Enter. Mándame la captura de lo que aparece.

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (develop)
● $ git branch
  * develop
    main
```

- 5.7. Modifica en local el index.html (ponle un cambio con algo descriptivo de lo que estás haciendo), confirma el cambio en el control de versiones local y haz un git push. Luego visita github y comprueba que el cambio que has hecho está solamente en la rama que creaste y que no aparece en la rama master.

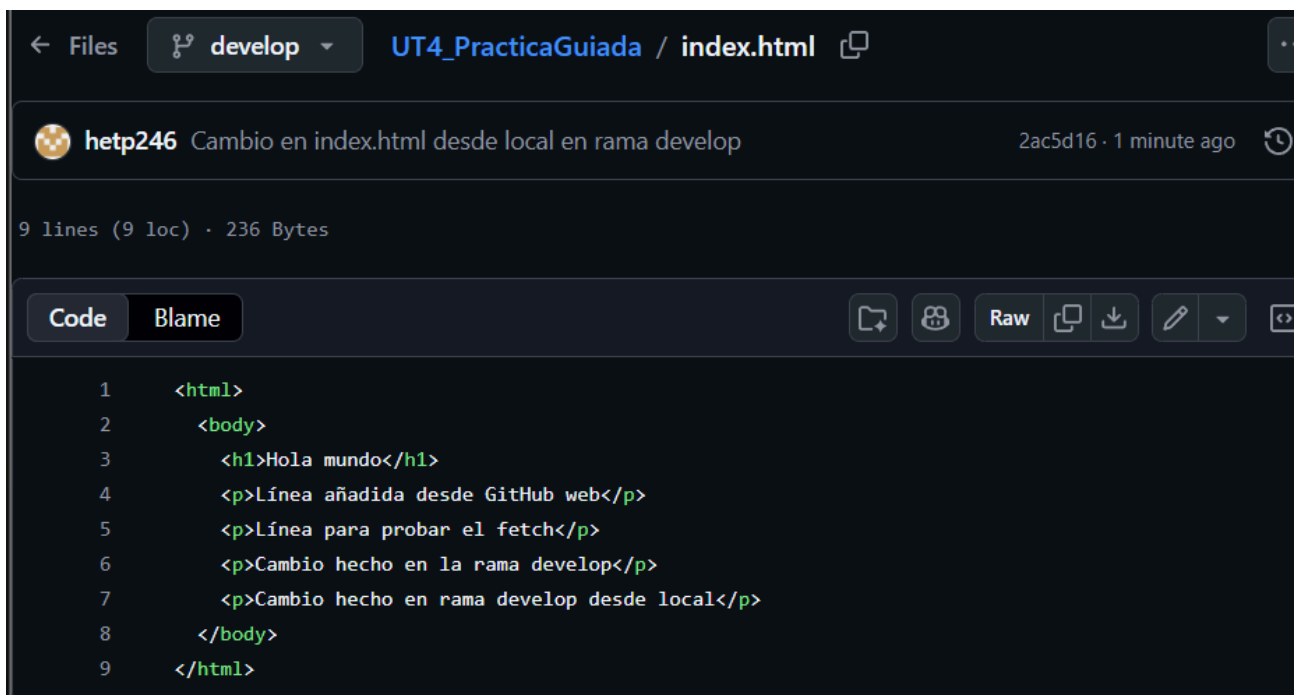
```
1  <html>
2    <body>
3      <h1>Hola mundo</h1>
4      <p>Línea añadida desde GitHub web</p>
5      <p>Línea para probar el fetch</p>
6      <p>Cambio hecho en la rama develop</p>
7      <p>Cambio hecho en rama develop desde local</p>
8    </body>
9  </html>
10
```

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (develop)
● $ git add .

vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (develop)
● $ git commit -m "Cambio en index.html desde local en rama develop"
[develop 2ac5d16] Cambio en index.html desde local en rama develop
1 file changed, 1 insertion(+)

vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (develop)
● $ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 343 bytes | 343.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/hetp246/UT4_PracticaGuiada.git
1052bbe..2ac5d16 develop -> develop

vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (develop)
● $
```



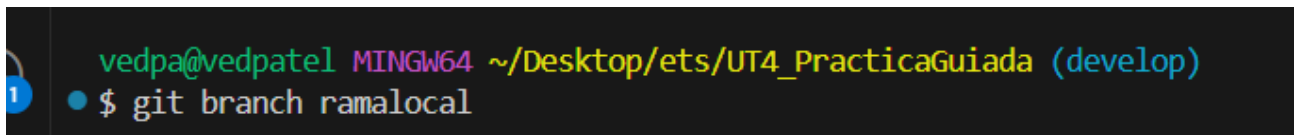
```
1 <html>
2 <body>
3   <h1>Hola mundo</h1>
4   <p>Línea añadida desde GitHub web</p>
5   <p>Línea para probar el fetch</p>
6   <p>Cambio hecho en la rama develop</p>
7   <p>Cambio hecho en rama develop desde local</p>
8 </body>
9 </html>
```

6. Hacer un push de una rama de Local hacia GitHub

Ahora vamos a ver como se puede hacer lo contrario, es decir crear una rama en nuestro repo local, y subirla al repo remoto.

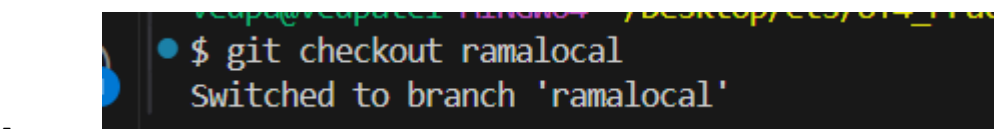
Para eso:

- 6.1. Crea una nueva rama en local (ya hemos visto como se hace con un `gitbranch <nombre>`). Adjunta captura con los comandos usados y lo que devuelven.



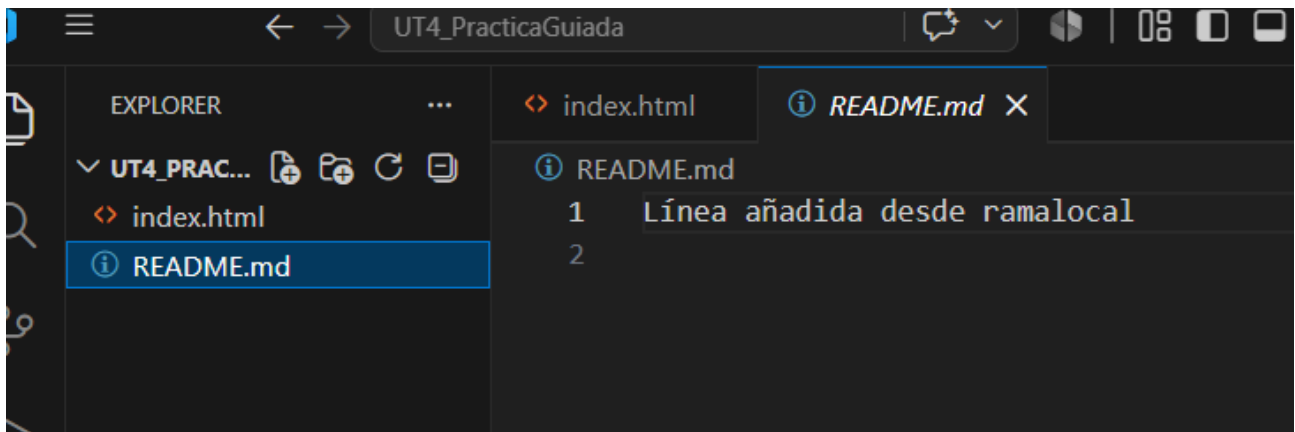
```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (develop)
$ git branch ramalocal
```

- 6.2. Muévete a esa rama (ya hemos visto como se hace con un `checkout`). Adjunta captura con los comandos usados.



```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (develop)
$ git checkout ramalocal
Switched to branch 'ramalocal'
```

- 6.3. Modifica el fichero `README.md` añadiendo lo que quieras en él. Confirma el cambio en el control de versiones local como hemos visto en la práctica anterior. Adjunta captura con los comandos usados.



```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (ramalocal)
• $ git add .
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (ramalocal)
• $ git commit -m "Modificación de README.md desde ramalocal"
[ramalocal f0f9fa8] Modificación de README.md desde ramalocal
1 file changed, 1 insertion(+)
create mode 100644 README.md

vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (ramalocal)
• $
```

- 6.4. Ejecuta el comando: `git push origin <nombre_de_la_rama_local_que_creaste>` Adjunta captura con lo que devuelve el comando.

```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (ramalocal)
• $ git push origin ramalocal
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 339 bytes | 339.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'ramalocal' on GitHub by visiting:
remote:   https://github.com/hetp246/UT4_PracticaGuiada/pull/new/ramalocal
remote:
To https://github.com/hetp246/UT4_PracticaGuiada.git
 * [new branch]      ramalocal -> ramalocal
```

- 6.5. Visita el repositorio en Github y comprueba que esté creada la nueva rama y que el contenido del README.md está actualizado a lo que pusiste. Adjunta captura donde se vea el nombre de la rama y la URL de tu repositorio (recuerda que de momento tu repo remoto debe estar puesta como público y no privado).

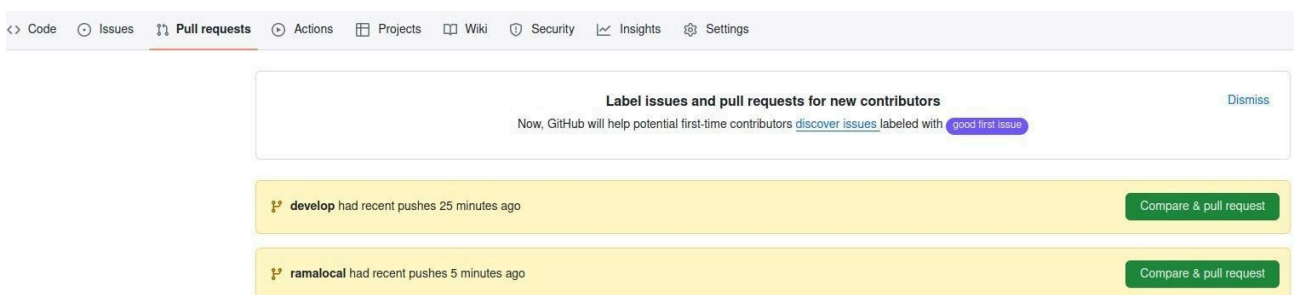


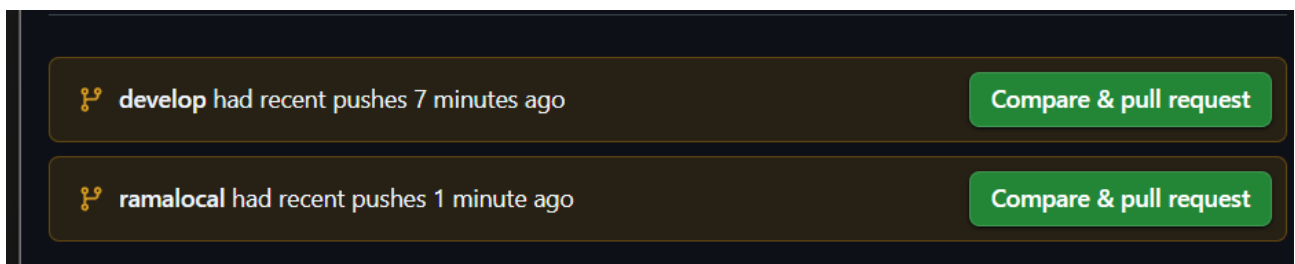
7. Pull requests y la fusión de ramas en GitHub

El concepto de **pull request** está relacionado con el de **aprobación de cambios**. Es decir hacer un pull request es hacer una **solicitud de actualización** de cambios (de fusión de ramas concretamente) al dueño del repositorio. Esos cambios tienen que ser aprobados para que se fusionen dos ramas.

Un Pull Request es una función de GitHub que permite a tu equipo solicitar la revisión y aprobación de sus cambios antes de fusionarlos en la rama principal de desarrollo (master).

Desde el servicio **web GitHub**, si vas a la sección de “pull request” verás información de los commits que se han hecho en cada rama y tienes opción incluso de comparar contenidos de ramas para fusionarlas si quieres y los aprobamos.

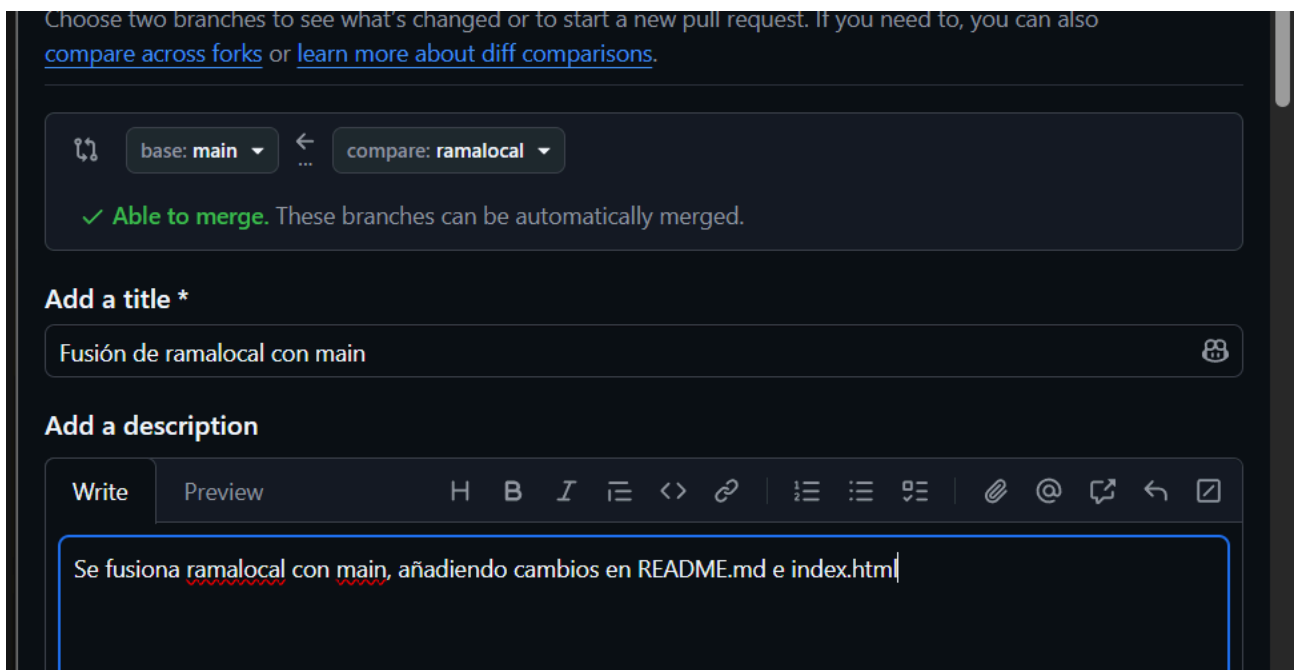




Si abres la opción de la pull request (“Compare & PullRequest”) verás información referente a las ramas que se supone que quieres fusionar:

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

A screenshot of the 'Open a pull request' form in GitHub. At the top, there are dropdown menus for 'base: master' and 'compare: ramalocal', followed by a green checkmark and the text 'Able to merge. These branches can be automatically merged.' Below this is a section titled 'Add a title' with a text input field containing 'Ramalocal'. To the right of the title field is a 'Reviewers' section with a 'No reviews' status and a settings icon.

 **Open** [hetp246](#) wants to merge 3 commits into `main` from `ramalocal` 

 Conversation **0**

 Commits **3**

 Checks **0**

 Files changed **2**



hetp246 commented [now](#)

Owner



Se fusiona ramalocal con main, añadiendo cambios en README.md e index.html



 **hetp246** added 3 commits [37 minutes ago](#)

  [Modificación de index.html en rama develop](#)  [1052bbe](#)

  [Cambio en index.html desde local en rama develop](#) [2ac5d16](#)

  [Modificación de README.md desde ramalocal](#) [f0f9fa8](#)



No conflicts with base branch

Merging can be performed automatically.

Merge pull request



You can also merge this with the command line. [View command line instructions.](#)

Fusión de ramalocal con main #1

 Merged

[hetp246](#) merged 3 commits into `main` from `ramalocal`  now

 Conversation 0

 Commits 3

 Checks 0

 Files changed 2



[hetp246](#) commented 1 minute ago

Owner

...

Se fusiona ramalocal con main, añadiendo cambios en README.md e index.html



[hetp246](#) added 3 commits 38 minutes ago



 [Modificación de index.html en rama develop](#) ...

[1052bbe](#)



 [Cambio en index.html desde local en rama develop](#)

[2ac5d16](#)



 [Modificación de README.md desde ramalocal](#)

[f0f9fa8](#)



 [hetp246](#) merged commit **6562699** into `main` [now](#)

Revert

Y te muestran los cambios que se han hecho y las líneas que se han añadido y quitado en cada fichero respecto a la rama **master**.

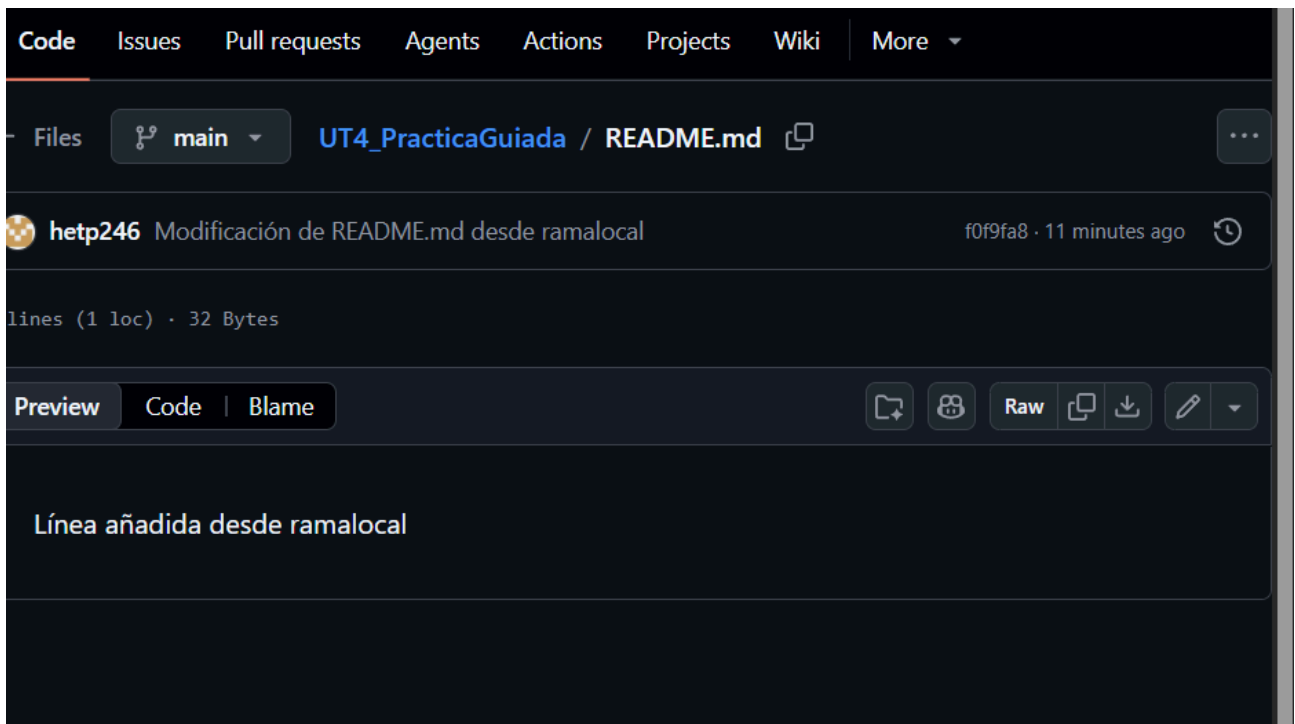
Showing 2 changed files with 4 additions and 1 deletion.

```
3 README.md
@@ -1,3 +1,4 @@
1 1 # hello-world
2 2 Esto es una prueba para empezar con nuestro
3 - primer repositorio local paso a paso.
3 + primer repositorio local paso a paso.
4 + <p>Linea para probar push a remoto desde ramalocal</p>

2 index.html
@@ -14,5 +14,7 @@ <h1>Hello world! Ahora que pasa </h1>
14 14
15 15 <p>Linea para probar el fetch</p>
16 16 <p>Linea para probar el fetch</p>
17 + <p>Linea para probar primer cambio en la rama develop</p>
18 + <p>Linea para probar segundo cambio en la rama develop desde local</p>
17 19 </body>
18 20 </html>
```

```
index.html
@@ -4,5 +4,6 @@ <h1>Hola mundo</h1>
4 <p>Línea añadida desde GitHub web</p>
5 <p>Línea para probar el fetch</p>
6 <p>Cambio hecho en la rama
develop</p>
7 + <p>Cambio hecho en rama develop desde
local</p>
7 </body>
8 </html>
8 </body>
9 </html>
```

- 7.1. Ve a la sección de pull request y crea una pull request para fusionar la última rama que has subido al repo, con la master. Echa un vistazo a los cambios que se van a hacer y ponle un buen nombre y una buena descripción. Adjunta captura de pantalla de todo el proceso.
- 7.2. Comprueba que tienes en la rama master el cambio que habías puesto en el fichero README.md, signo de que se han fusionado correctamente las dos ramas.



- 7.3. Ahora vamos a generar un conflicto entre ramas del repo remoto. Para ello sigue estos pasos:
 1. En GitHub seleccionando la rama master como la de trabajo, modifica el fichero index.html desde el editor (añade nuevas líneas con algo descriptivo).

```
Edit Preview

1  <html>
2    <body>
3      <h1>Hola mundo</h1>
4      <p>Línea añadida desde GitHub web</p>
5      <p>Línea para probar el fetch</p>
6      <p>Cambio hecho en la rama develop</p>
7      <p>Cambio hecho en rama develop desde local</p>
8      <p>Cambio en main para generar conflicto</p>
9    </body>
10 </html>
11
```

Commit changes

Commit message

Cambio en main para conflicto

Extended description

Add an optional extended description...

Message and description suggested by Copilot.

☒ Commit directly to the main branch

☐ Create a new branch for this commit and start a pull request [Learn more about pull requests](#)

Cancel Saving...

2. A continuación cambia a la otra rama que creaste y modifica el fichero index.html con otras modificaciones distintas a la anterior.

i Hise lo mismo que ariva pero con develop

3. Vuelve a cambiar a la rama master y ve a sección de “pull request”. Deberás ver algo parecido a esto:

develop had recent pushes less than a minute ago

Compare & pull request

develop had recent pushes 32 seconds ago

Compare & pull request

Haz click en la opción de “Compare & Pull Request” y echa un vistazo a lo que te presenta. Comenta lo que ves, adjunta capturas haz el pull request. ¿Hay algún conflicto? Resuélvelo. Comenta lo que has hecho para resolver el conflicto y adjunta capturas.

Add a title *

Fusión develop con main - conflicto

Add a description

Write Preview H B I ≡ <> 🔗 | ≡ ≡ ≡ | 📎 @ ↗ ↶ ↷

Se intenta fusionar develop con main generando un conflicto en index.html

Markdown is supported Paste, drop, or click to add files

Create pull request

- 7.4. Para acabar, vamos a generar conflictos entre nuestro repo local y el repo remoto. Para ello sigue estos pasos:

1. Actualiza el repo remoto desde github añadiendo por ejemplo la siguiente línea al fichero index.html:

`<p>CONFLICTO REMOTO</p>`

```
<p>cambio en main para generar conf  
<p>CONFLICTO REMOTO</p>  
</body>
```

Commit changes

Commit message

Conflicto remoto

Extended description

Add an optional extended description...

☒ Commit directly to the `main` branch

☐ Create a **new branch** for this commit and start a pull request
[more about pull requests](#)

Cancel Save

2. Actualiza el repo local desde la terminal añadiendo por ejemplo la siguiente línea al fichero `index.html`:

`<p>CONFLICTO LOCAL</p>`

```

8 | <p>CONFLICTO LOCAL</p>
9 | </body>

```

3. Trata de hacer un push para actualizar el repo remoto desde la terminal. Adjunta captura del mensaje de devuelve.

Ejecuta un pull para actualizar tu repo local y comenta lo que ocurre ¿Hay conflictos? Utiliza la ayuda que te ofrece el VS Code para detectar y corregir los conflictos. En este caso lo que se pide para resolver los conflictos es que te quedes con los últimos cambios de la rama remota y los últimos tuyos de la rama local.

```

vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (ramalocal)
• $ git add .

vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (ramalocal)
• $ git commit -m "Conflicto local"
[ramalocal a44c12e] Conflicto local
1 file changed, 1 insertion(+)

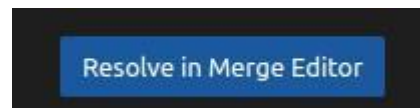
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (ramalocal)
• $ git checkout main
Switched to branch 'main'
Your branch is behind 'origin/main' by 6 commits, and can be fast-forwarded.
(use "git pull" to update your local branch)

```



```
vedpa@vedpatel MINGW64 ~/Desktop/ets/UT4_PracticaGuiada (main)
● $ git pull
Updating 3cae953..ea38aeb
Fast-forward
 README.md | 1 +
 index.html | 4 +++
 2 files changed, 5 insertions(+)
 create mode 100644 README.md
```

- 7.5. Adjunta captura de la interfaz que te ofrece VS Code para resolver los conflictos cuando haces click en la opción "Resolve in Merge Editor".



Intenta entender y explica la ayuda que visual de VS Code te ofrece (tienes el editor dividido en 3 espacios, explica qué indica cada uno) y que has hecho para resolver el conflicto.

IMPORTANTE:

Para la entrega de esta práctica se tiene que subir **un solo fichero PDF** que contenga:

- **Memoria** de la práctica donde se resuelven las actividades, se contesta a las preguntas, etc. La memoria se ha de subir en formato .PDF.

En la practica se debe incluir la URL del Repositorio de GiitHub Creado y éste debe ser público!!!! (Apartado 1.1)

El nombre de la memoria debe seguir el siguiente **formato**:

UT4_PracticaGuiada_6_NombreYApellidosDelAlumno.pdf

(donde Nombre_Apellidos debe ser el nombre y apellido/s del alumno)